

AMDP 練習 5：錯誤處理與例外

Lecture

前面四題的 AMDP Method 都是「一定會成功」的查詢，沒有處理過「SQLScript 內部發現問題，該怎麼通知呼叫端」這件事。承 OOP op09 教過的自訂例外類別設計，這題示範 AMDP 這一側對應的機制：**SQLScript 用 SIGNAL 主動拋出一個錯誤**，這個錯誤會被 **AMDP 框架包裝成 ABAP 的 CX_AMDP_ERROR 例外**，呼叫端用一般的 **TRY...CATCH** 攔截。

這跟 **ABAP 的 RAISE EXCEPTION** 語意上是同一件事，只是發生的位置不同：**RAISE EXCEPTION** 是 ABAP 程式碼在 ABAP 應用伺服器裡拋出例外；**SIGNAL** 是 SQLScript 程序在 HANA 資料庫引擎裡拋出錯誤——但因為 AMDP 把「呼叫資料庫程序」包裝成「呼叫一個 ABAP Method」，資料庫端的錯誤最終也會用 ABAP 熟悉的例外機制 (**CX_AMDP_ERROR**) 浮現給呼叫端，不需要呼叫端自己去解析什麼奇怪的資料庫錯誤碼。

重要限制：這是單向的——AMDP 的 SQLScript 本體沒有辦法「呼叫回」ABAP 端的邏輯（不能在 SQLScript 裡呼叫一個 ABAP Method 或 Function Module），錯誤處理的方向永遠是「SQLScript 拋錯 → ABAP 接住」，不會有反過來的情況。這也呼應 am01 開始就強調的定位：AMDP 是「ABAP 呼叫資料庫」的機制，資料庫端執行時是獨立、封閉的環境。

學習目標

- 會在 SQLScript 裡用 **DECLARE <條件名> CONDITION FOR SQL_ERROR_CODE <代碼>**；宣告一個具名的錯誤條件，並用 **SIGNAL <條件名> SET MESSAGE_TEXT = '...'**；主動拋出
- 理解 AMDP Method 若要拋出例外，ABAP 端簽章要宣告 **RAISING cx_amdp_error**，呼叫端用 **TRY...CATCH cx_amdp_error** 攔截（跟 op09 教過的自訂 **ZCX_** 例外用法幾乎一樣，只是這裡攔截的是框架提供的 **CX_AMDP_ERROR**，不是自己定義的例外類別）
- 知道 **CX_AMDP_ERROR->GET_TEXT()** 拿到的訊息，是 HANA 資料庫回傳的**原始技術性錯誤訊息**（包含程序名稱、行號、位置等診斷資訊），自訂的錯誤文字會被包在這一大串技術訊息的最後面，不是一個乾淨獨立的訊息——直接把這個訊息原封不動顯示給終端使用者不太合適，通常要自己解析或只在技術記錄（log）裡用
- 理解 AMDP 是單向的：SQLScript 可以拋錯給 ABAP 接，但 SQLScript 沒辦法反過來呼叫 ABAP 的邏輯

事前準備

ADT 端物件已由課程準備好（\$TMP）：

- **ZCL_AM05_FLIGHT_VALIDATOR**——AMDP 類別，**check_carrier_exists** 方法檢查 **carrid** 是否存在於 **SCARR**，不存在就用 **SIGNAL** 拋出錯誤
- **ZR_AM05_DEMO**——demo 程式，測試一個存在的 **carrid**（**LH**）跟一個不存在的（**ZZ**），用 **TRY...CATCH cx_amdp_error** 攔截並印出錯誤訊息

題目需求（對照已建好的答案物件，含實測踩坑過程）

1. **ZCL_AM05_FLIGHT_VALIDATOR** 的簽章與 SQLScript 本體：

```
``abap CLASS zcl_am05_flight_validator DEFINITION PUBLIC FINAL CREATE PUBLIC. PUBLIC SECTION.
INTERFACES if_amdp_marker_hdb.
```

```
CLASS-METHODS check_carrier_exists IMPORTING VALUE(iv_mandt) TYPE mandt VALUE(iv_carrid) TYPE
s_carr_id RAISING cx_amdp_error. ENDCLASS.
```

```
CLASS zcl_am05_flight_validator IMPLEMENTATION.
```

```
METHOD check_carrier_exists BY DATABASE PROCEDURE FOR HDB LANGUAGE SQLSCRIPT OPTIONS READ-
ONLY USING scarr.
```

```
DECLARE lv_cnt INTEGER; DECLARE SQLSCRIPT_ERROR CONDITION FOR SQL_ERROR_CODE 10001;
```

```
SELECT COUNT(*) INTO lv_cnt FROM scarr WHERE mandt = :iv_mandt AND carrid = :iv_carrid;
```

```
IF :lv_cnt = 0 THEN SIGNAL SQLSCRIPT_ERROR SET MESSAGE_TEXT = 'Carrier not found in SCARR'; END IF;
```

```
ENDMETHOD.
```

```
ENDCLASS. ``
```

1. 實測踩坑：SIGNAL SQLSCRIPT_ERROR ... 沒有先宣告，啟用失敗：

第一版直接用 SIGNAL SQLSCRIPT_ERROR SET MESSAGE_TEXT = '...';，以為 SQLSCRIPT_ERROR 是 SQLScript 內建、隨時可用的通用錯誤條件（有些教材範例是這樣示範的），結果啟用時 HANA 編譯器回報：SQLSCRIPT message: identifier must be declared: SQLSCRIPT_ERROR——這套系統版本要求任何要 SIGNAL 的條件名稱，都必須先用 DECLARE <名稱> CONDITION FOR SQL_ERROR_CODE <代碼>; 明確宣告，不能憑空拿一個名字就 SIGNAL。這題選用 SQL_ERROR_CODE 10001（AMDP 官方文件慣例上常用的自訂錯誤碼區間），宣告完成後 SIGNAL SQLSCRIPT_ERROR SET MESSAGE_TEXT = '...'; 才能正常運作。

1. ZR_AM05_DEMO 呼叫端：

```
``abap REPORT zr_am05_demo LINE-SIZE 250.
```

```
DATA(lt_test_carriers) = VALUE string_table( ( LH ) ( ZZ ) ).
```

```
LOOP AT lt_test_carriers INTO DATA(lv_carrid_str). DATA(lv_carrid) = CONV s_carr_id( lv_carrid_str ).
```

```
TRY. zcl_am05_flight_validator=>check_carrier_exists( iv_mandt = sy-mandt iv_carrid = lv_carrid ). WRITE: /
lv_carrid, ': OK, carrier exists'. CATCH cx_amdp_error INTO DATA(lx_error). WRITE: / lv_carrid, ': ERROR -',
lx_error->get_text( ). ENDTRY. ENDLOOP. ``
```

跟一般攔截自訂 ZCX_ 例外的寫法完全一樣（呼應 op09），差別只在這裡攔截的是框架提供的 CX_AMDP_ERROR。

預期輸出（實測畫面）

```
LH : OK, carrier exists
ZZ : ERROR - Error when executing the database procedure
      "ZCL_AM05_FLIGHT_VALIDATOR=>CHECK_CARRIER_EXISTS".
      SQL error: "10,001". SQL message: "user-defined error: ...
      ...Carrier not found in SCARR".
```

LH 存在，正常通過；ZZ 不存在，**SIGNAL** 觸發、被 **CATCH cx_amdp_error** 接住，**get_text()** 印出的訊息開頭是 AMDP 框架的標準包裝文字（含程序名稱、SQL 錯誤碼 **10,001**——剛好對應 **DECLARE ... FOR SQL_ERROR_CODE 10001** 宣告的代碼），自訂的 **'Carrier not found in SCARR'** 文字則接在整段訊息的最後面（實測整段訊息長度 379 字元，遠比自訂文字本身長很多）。

團隊實務備註

- **CX_AMDP_ERROR->GET_TEXT()** 的內容是「除錯用的技術訊息」，不是「使用者看得懂的錯誤訊息」：這題實測整段訊息包含程序全名、內部版本戳記（**#stb2#20260719140914** 這種）、行號位置（**line 9 col 3**、**line 16 col 7**）等對終端使用者毫無意義的資訊，自訂文字只是其中一小段、還排在最後面。如果要把這個錯誤呈現給使用者（例如包成 REST API 的 400 錯誤訊息，呼應 REST 課程 rs07 的統一錯誤格式），應該自己額外定義商業邏輯層的判斷（例如程式先自己查一次 **SCARR** 判斷存不存在、用自己的 **ZCX_** 例外拋出乾淨的訊息），而不是把 **CX_AMDP_ERROR** 的原始訊息直接丟給使用者
- **DECLARE ... CONDITION FOR SQL_ERROR_CODE <代碼>** 的代碼是你自己選的，這題選 **10001** 只是照著常見 AMDP 教材的慣例（HANA 系統保留碼之外、自訂錯誤常用的區間），同一個 AMDP 類別如果要拋出多種不同情境的錯誤，可以宣告多個不同代碼的條件，呼叫端可以從 **SQL error: "10,001"** 這段文字分辨是哪一種錯誤（雖然要自己字串比對，不像 ABAP **ZCX_** 例外可以用 **IF_T100_MESSAGE** 那樣結構化）
- 這題選擇「檢查完直接 **SIGNAL**」而不是「檢查完回傳一個狀態旗標，讓 ABAP 端自己判斷要不要 **RAISE EXCEPTION**」：兩種設計都合理，各有取捨——**SIGNAL** 讓錯誤處理邏輯完全留在 SQLScript 裡（呼叫端只要單純 **TRY...CATCH** 就好，不用自己判斷回傳值），但換來的是錯誤訊息比較不乾淨（如上一點所述）；回傳狀態旗標讓 ABAP 端能用自己的 **ZCX_** 例外包出乾淨訊息，但每個檢查點都要多一個輸出參數，介面變得囉唆。實務上常見做法是兩者混用：AMDP 內部單純的資料驗證用回傳旗標，真正意外、不該發生的資料庫層級錯誤才讓它自然拋出變成 **CX_AMDP_ERROR**

思考題

1. 如果 **check_carrier_exists** 改成回傳一個 **ev_exists TYPE abap_bool** 的 **EXPORTING** 參數，而不是用 **SIGNAL**，讓呼叫端自己判斷要不要 **RAISE EXCEPTION TYPE zcx_...**，這樣的錯誤訊息會不會比較乾淨？你會怎麼取捨這兩種設計？
2. 這題的 **DECLARE SQLSCRIPT_ERROR CONDITION FOR SQL_ERROR_CODE 10001**；如果換成另一個代碼（例如 **20001**），呼叫端攔截 **cx_amdp_error** 的程式碼需要跟著改嗎？（提示：**CATCH cx_amdp_error** 攔截的是例外的**型別**，不是特定的 SQL 錯誤碼，型別攔截不用管代碼是多少，但要分辨「是哪一種錯誤」就得自己解析 **get_text()** 裡的代碼字串）
3. 如果同一個 AMDP 方法在 SQLScript 裡 **SIGNAL** 之後，後面還有其他 SQLScript 語句（例如原本想在檢查完之後再做一次查詢），這些語句還會執行嗎？**SIGNAL** 對 SQLScript 程序本身的執行流程有什麼影響？（提示：可以類比 ABAP 的 **RAISE EXCEPTION** 之後，同一個 Method 裡後面的程式碼還會不會繼續跑）

答案

見 **zcl_am05_flight_validator.clas.abap**、**zr_am05_demo.prog.abap**（SAP 端物件 **ZCL_AM05_FLIGHT_VALIDATOR / ZR_AM05_DEMO**，套件 **\$TMP**）。